

Confluent Kafka Isotope

An example of using Kafka **consumer and producer interceptors** to tag every message with a tracer — an *isotope* — that carries through every hop of a multi-topic pipeline. Apache Flink consumes the tagged records and reports on what the isotopes reveal: end-to-end latency, hop topology, drop/duplication rates, and pipeline coverage.

A **portability requirement** runs through this project: the same isotope mechanism must work against both **Confluent Cloud for Apache Flink** (managed; restricted to Table API SQL + uploaded UDF/PTF JARs) and **Confluent Platform for Apache Flink** (full DataStream + Table API). Three decisions follow from that: tagging happens in the Kafka **producer/consumer interceptors** (the one extension point both runtimes share via the broker), the on-wire **header** format is **JSON** (so Flink SQL can read the scalar fields with `CAST(headers[...] AS STRING)` and no UDF), and the optional stateful reports (`LatencyPercentilesUDAF`, `StuckTracePTF`) ship as a single JAR that registers identically on either runtime.

Message **values** on the demo topics are **SR-framed Protobuf** (`ai.signalroom.kafka.isotope.proto.DemoEvent`) — the standard Confluent value format. The interceptors and reports are agnostic to value format because the isotope rides in headers; the Protobuf choice just gives the integration tests and the demo CLI a typed payload to work with.

How the isotope is carried

- **Header `x-isotope`** (JSON bytes) carries the full hop history, forwarded by every hop:
 - `t` — 16-byte **UUIDv7** trace ID (RFC 9562): 48-bit ms timestamp in the high bits + 74 bits random. Stable for the life of the trace, and lexicographic byte order matches creation order — sort trace IDs and you get chronological order for free.
 - `o` — origin timestamp (ms) — same value as the timestamp embedded in the UUIDv7 trace ID; kept as its own field for typed access from Flink SQL without needing to decode the UUID bytes.
 - `s` — origin service name (set once, never reassigned)
 - `h` — ordered list of hops, each `{s: service, t: topic, m: tsMs}`
 - `x` — `true` if the hop list exceeded `MAX_HOPS = 32` and the oldest hop was evicted
- **Six scalar headers** (UTF-8 strings) carry the most-recent-hop view so Flink SQL can read them via `CAST(headers['x-isotope-...'] AS STRING)` without parsing the JSON array (no UDF required on either CCAF or CP Flink). See [flink/README.md](#) for the full header table.

A producer with the isotope interceptor loaded appends one hop on every `send()`. A consume-then-produce service calls `IsotopeContext.adoptFromRecord(record)` between consume and produce so the trace ID and origin survive the hop.

Repo layout

```
app/
tests
src/main/proto/
```

```
isotope JVM library + demo CLI +
DemoEvent.proto (Protobuf message
```

```

value)
  src/main/java/ai/signalroom/kafka/isotope/
    Isotope.java                POJO + JSON codec + Hop +
fromHeaders()                  + UUIDv7 helpers (uuidV7Bytes /
                                + UUIDv7 helpers (uuidV7Bytes /
uuidV7String)
  IsotopeContext.java           ThreadLocal + adoptFromRecord()
  IsotopeProducerInterceptor.java stamps/appends x-isotope + 6 scalar
                                reporting headers on send()
                                batch-aware logging; no auto-
  IsotopeConsumerInterceptor.java propagation
  App.java                      demo CLI – send / hop / sink modes
  src/test/java/.../            IsotopeCodecTest (no broker needed)
  src/integrationTest/java/.../ live-broker tests; produce/consume
                                DemoEvent via SR-framed Protobuf
                                (need Minikube CP + SR port-
forwarded)
ptf/                            Phase 2 – Flink PTF + UDAF shadow
JAR
  src/main/java/ai/signalroom/kafka/isotope/flink/
    LatencyPercentilesUDAF.java T-Digest p50/p95/p99 aggregate
    StuckTracePTF.java          per-trace state + event-time timer
    Percentiles.java, StuckTraceAlert.java return-type DT0s
  src/test/java/.../            LatencyPercentilesUDAFTest
flink/sql/                      Flink SQL reports – identical on
                                CCAF and CP Flink except source DDL
                                per-environment source + shared
  cp/, cc/, shared/
reports
k8s/base/                       CFK Kafka/SR/Connect/ksqlDB/C3
manifests
scripts/                        port-forward helpers, deploy-flink-
reports.sh
Makefile                        cp-up / flink-up / kafka-pf-up /
...

```

Running

1. Unit tests (no broker, instant)

```

./gradlew test                  # both subprojects
# or scoped:
./gradlew :app:test             # IsotopeCodecTest – JSON
roundtrip, hop eviction, header size, UUIDv7 properties (10 tests)
./gradlew :ptf:test            # LatencyPercentilesUDAFTest – T-
Digest accumulator semantics

```

2. Demo CLI — see one trace propagate live

The fastest way to watch the isotope mechanic. Requires the cluster to be up and the Kafka + SR forwards running (see step 3 below for the bring-up commands). The CLI has three modes:

Mode	Args	What it does
send	<topic> <service> <payload>	Produces one isotope-tagged DemoEvent to <topic>, then exits. Auto-creates the topic.
hop	<in-topic> <out-topic> <service>	Consumes records from <in-topic>, adopts the isotope into thread-local, re-produces the same DemoEvent to <out-topic> as <service> (which appends a new hop). Runs until Ctrl-C.
sink	<topic>	Subscribes to <topic> and pretty-prints the full isotope trail for every arriving record. Runs until Ctrl-C.

A 3-stage chain in four terminals:

```
# Terminal A – terminal sink (will print the full 3-hop trail)
./gradlew :app:run --args="sink iso-final" -q

# Terminal B – middle stage: iso-mid → iso-final as svc-C
./gradlew :app:run --args="hop iso-mid iso-final svc-C" -q

# Terminal C – first stage: iso-start → iso-mid as svc-B
./gradlew :app:run --args="hop iso-start iso-mid svc-B" -q

# Terminal D – kick the chain off (run repeatedly to send more)
./gradlew :app:run --args="send iso-start svc-A 'hello world'" -q
```

Terminal A's output for each record shows the same **trace_id** across all three hops, **origin = svc-A** (never reassigned), and **hops[]** listing **svc-A → svc-B → svc-C** in order with per-hop timestamps. Override endpoints via **-Dkafka.bootstrap=...** / **-Dschema.registry.url=...** if you're not on the default Minikube layout.

3. Integration tests (live Kafka via Minikube)

Bring up the local Confluent Platform stack and port-forward Kafka + SR:

```
make minikube-start          # one-time
make cp-up                   # CFK Operator +
Kafka/SR/Connect/ksqlDB/C3 (~5 min)
make kafka-pf-up             # localhost:30092 → Kafka,
localhost:8081 → Schema Registry
```

Then run the suite:

```
./gradlew :app:integrationTest #
all 5 tests
```

```
./gradlew :app:integrationTest --tests '*ProducerInterceptorIT' #
just one
```

Override the endpoints if needed:

```
./gradlew :app:integrationTest \
  -PkafkaBootstrap=localhost:30092 \
  -PschemaRegistryUrl=http://localhost:8081
```

Tear down forwards when done:

```
make kafka-pf-down
```

The integration tests cover:

Test	What it verifies
BrokerSmokeIT	AdminClient can create/list/delete a topic via the NodePort port-forward
ProducerInterceptorIT	A bare consumer sees the x-isotope JSON header + all 6 scalar reporting headers with the expected origin/hop values, and the Protobuf round-trip preserves DemoEvent.source / payload
ConsumerInterceptorIT	IsotopeContext.adoptFromRecord extracts isotope into thread-local on tagged records; clears the thread-local for untagged records
ThreeStageHopPropagationIT	svc-A → topic-AB → svc-B → topic-BC → svc-C produces a stable trace ID, 2-hop trail in send order, and correct scalar headers (origin = svc-A , this = svc-B , hop count = 2) at the terminal

4. Flink SQL reports on Minikube

The four Phase-1 reports plus the two Phase-2 (PTF / UDAF) reports run against a Flink session cluster managed by the Confluent Flink Kubernetes Operator. Same FQL files deploy to Confluent Cloud for Apache Flink — see [flink/README.md](#) for that side; this section is the local-Minikube path.

Bring up Flink:

```
make flink-up          # cert-manager → CFK Flink Operator → CMF →
session cluster        # (~5 min the first time)
```

Deploy the reports — there are now **two deployment paths**, because CMF supports SR-Protobuf natively but disallows user-defined functions (and the percentiles report uses a T-Digest UDAF):

Reports	Path	Where it lives
latency, topology, hop_distribution, coverage	CMF Statements (canonical)	Each its own Application-mode Flink cluster; SR-Protobuf auto-registered; Control Center renders natively
latency_percentiles_flat	cp-flink session cluster (UDAF-only)	Shares the open-source FlinkDeployment; uses Apache Flink's protobuf format (no SR magic byte)

```
make cmf-reports-up           # 4 reports on CMF (SR-Protobuf)
make flink-reports-up        # latency_percentiles only on cp-flink
session
```

`cmf-reports-up` creates a CMF KafkaCatalog (wrapping SR), a KafkaDatabase (wrapping Kafka), a ComputePool, then `ALTER TABLE iso-start ADD headers METADATA ...` (so the auto-exposed source has the `x-isotope-*` headers), and finally submits 4x(DDL, INSERT INTO) Statements. CMF auto-creates each `isotope-report-*-1m` Kafka topic and registers a Protobuf schema in SR for the `*-value` subject. **Control Center deserializes those topics natively** — no .proto schemas in the repo, no hand-installed format jars, no version juggling.

`flink-reports-up` is now scoped to the **latency_percentiles report only**. It still uses Apache Flink's open-source `protobuf` format (no SR magic byte) on the `isotope-report-latency-percentiles-1m` topic, with the `LatencyPercentilesUDAF` T-Digest aggregate from `ptf/src/main/java/.../LatencyPercentilesUDAF.java`. Control Center can't decode that topic without an out-of-band `.proto` (`ptf/src/main/proto/.../reports.proto`). The downstream demo topics (`iso-start`, etc.) still use SR-Protobuf via the Java app; that's unchanged.

Why the split?

CMF 2.3.1 has a clean docs-supported flow for SR-Protobuf — but it **disallows UDFs in Statements** ([features-support page](#)). The 4 portable reports (pure SQL aggregates) migrate cleanly. The percentiles report depends on a custom T-Digest UDAF and stays on the session cluster where UDFs work. Two paths is the honest answer on this stack.

Version note

CMF's compute pool runs the `confluentinc/cp-flink-sql:1.19-cp8-arm64` image — Confluent has not yet published a 2.x build of the `cp-flink-sql` image (which bundles `ce-flink-sql-job.jar` for CMF Statement execution). The cp-flink session cluster runs Flink 2.1.2. Two Flink versions in the same project. They're isolated (different pods, different JVMs, only share Kafka + SR), so the mismatch is ergonomic, not load-bearing.

Results persist in Kafka, not in the SQL Client session. The streaming **INSERT INTO** jobs run on the Flink session cluster indefinitely, accumulating one row per closed window per group into their sink topic. Closing your SQL Client session does not stop them — use **make flink-reports-down** for that.

Query interactively — opens a Flink SQL Client inside the JobManager pod. If **flink-reports-up** has already run, the sink table DDL is auto-loaded so **SELECT *** works out of the box:

```
make flink-sql
# Flink SQL> SELECT * FROM latency_report_1m;
# Flink SQL> SELECT * FROM topology_report_1m;
# Flink SQL> SELECT * FROM hop_distribution_1m;
# Flink SQL> SELECT * FROM coverage_report_1m;
# Flink SQL> SELECT * FROM latency_percentiles_flat_1m;      --
Phase 2 UDAF
# NOTE: `stuck_trace_alerts` (Phase 2 PTF) is not deployed on the
#       Minikube cluster – see the caveat below.
```

Each **SELECT** here is a *read-only* tail of the sink topic — it doesn't start an aggregation job, just streams whatever the already-running INSERTs have produced. Closed-window rows accumulate over time, so a fresh **SELECT *** shows the full history (until the topic's retention policy kicks in).

The Flink UI shows the 5 long-lived INSERT jobs (one per report) plus any ad-hoc SELECTs you start:

```
make flink-ui          # opens http://localhost:8081 in your browser
```

Drive traffic in another terminal while the queries are running:

```
./gradlew :app:run --args="send iso-start svc-A 'hello'" -q
# or the full 3-stage chain from § 2 above
```

You should see report rows update as records flow.

Teardown — cancels the running INSERT INTO jobs via the Flink REST API, drops the catalog tables / views / functions, **and deletes the isotope-report-*-1m Kafka topics** (all historical report data is lost):

```
make flink-reports-down
```

Known caveat: STUCK_TRACE_PTF does not deploy on this Flink image

PTFs in Confluent Flink are an **Early Access Program** feature, and their SQL call syntax is in flux. The stuck-trace PTF view uses the Confluent-documented form — **input => TABLE isotope PARTITION BY trace_id, on_time => DESCRIPTOR(...)** — which **the parser shipped in confluentinc/cp-**

flink:2.1.1-cp1-java21 rejects with **Encountered "PARTITION"** at line **N**, column **M**. We confirmed this by walking four syntactic variants (named-arg + bare TABLE, named-arg + parenthesised subquery, positional + named hybrid, all-named); all hit the same grammar gate or its Calcite **ClassCastException** downstream.

What I'm **not** sure about: whether the parser in this image is genuinely behind CCAF's, whether there's an opt-in flag for the EAP grammar, or whether the docs run ahead of any current Flink build. The Confluent EAP-feature page says this syntax works on CCAF; we haven't verified that directly on a CCAF cluster.

What this means in practice:

- **make flink-reports-up** applies **8 of 9** reports successfully on Minikube; **60_stuck_trace_report.fql** is intentionally skipped by the deploy script and carries a banner comment explaining why.
- The PTF Java code in [ptf/src/.../StuckTracePTF.java](#) matches the canonical Confluent [InactivityAlert example](#) structurally, so the JAR should be portable when the parser side catches up (or against CCAF directly).
- The UDAF (**LATENCY_PERCENTILES**) is unaffected — UDAFs don't use PTF-call syntax, and **latency_percentiles_flat_1m** deploys fine on Minikube.

Recommended path the first time through

1. **./gradlew test** — proves the codec + UDAF logic without any cluster.
2. **make cp-up && make kafka-pf-up && ./gradlew :app:integrationTest** — proves the broker + SR + interceptor + Protobuf path end-to-end.
3. The 3-stage demo CLI walkthrough above — visually shows the trace accumulating hops.
4. **make flink-up && make flink-reports-up && make flink-sql** — reports populate as you drive traffic via the demo CLI (see § 4).

Status

Piece	Status
Kafka interceptors + JSON codec	Implemented; 10 unit tests passing (codec roundtrip, hop eviction, header size, UUIDv7 version/variant/order properties)
Live-broker integration tests	All 5 tests passing against Minikube CP (Kafka 4.x + SR 8.2.0)
Demo CLI (send / hop / sink)	Implemented and exercised end-to-end
Protobuf message values via SR	Implemented; integration tests round-trip DemoEvent
Flink SQL reports — Phase 1 (4 reports)	make flink-reports-up deploys all 4 (latency_report_1m , topology_report_1m , hop_distribution_1m , coverage_report_1m) on Minikube
Flink UDAF — Phase 2 (LATENCY_PERCENTILES)	Shadow JAR builds (89 KB), UDAF unit-tested (5 tests), deploys + queries on Minikube via latency_percentiles_flat_1m

Piece	Status
Flink PTF — Phase 2 (<i>STUCK_TRACE_PTF</i>)	Java code complete and unit-compileable; SQL call deferred — Apache Flink 2.1.1's parser rejects the documented EAP PTF call syntax (see § 4 caveat). JAR + FQL ready for CCAF when validated there.